

Let's see if we can come up with a representation. We know run needs to execute asynchronous tasks somehow. We could write our own low-level API, but there's already a class that we can use in the Java Standard Library, `java.util.concurrent.ExecutorService`. Here is its API, excerpted and transcribed to Scala:

```
class ExecutorService {
  def submit[A](a: Callable[A]): Future[A]
}
trait Callable[A] { def call: A }
trait Future[A] {
  def get: A
  def get(timeout: Long, unit: TimeUnit): A
  def cancel(evenIfRunning: Boolean): Boolean
  def isDone: Boolean
  def isCancelled: Boolean
}
```

 Essentially just a lazy A

So `ExecutorService` lets us submit a `Callable` value (in Scala we'd probably just use a lazy argument to submit) and get back a corresponding `Future` that's a handle to a computation that's potentially running in a separate thread. We can obtain a value from a `Future` with its `get` method (which blocks the current thread until the value is available), and it has some extra features for cancellation (throwing an exception after blocking for a certain amount of time, and so on).

Let's try assuming that our `run` function has access to an `ExecutorService` and see if that suggests anything about the representation for `Par`:

```
def run[A](s: ExecutorService)(a: Par[A]): A
```

The simplest possible model for `Par[A]` might be `ExecutorService => A`. This would obviously make `run` trivial to implement. But it might be nice to defer the decision of how long to wait for a computation, or whether to cancel it, to the caller of `run`. So `Par[A]` becomes `ExecutorService => Future[A]`, and `run` simply returns the `Future`:

```
type Par[A] = ExecutorService => Future[A]
```

```
def run[A](s: ExecutorService)(a: Par[A]): Future[A] = a(s)
```

Note that since `Par` is represented by *a function* that needs an `ExecutorService`, the creation of the `Future` doesn't actually happen until this `ExecutorService` is provided.

Is it really that simple? Let's assume it is for now, and revise our model if we find it doesn't allow some functionality we'd like.

7.3 Refining the API

The way we've worked so far is a bit artificial. In practice, there aren't such clear boundaries between designing the API and choosing a representation, and one doesn't necessarily precede the other. Ideas for a representation can inform the API, the API can inform the choice of representation, and it's natural to shift fluidly between these two perspectives, run experiments as questions arise, build prototypes, and so on.

We'll devote this section to exploring our API. Though we got a lot of mileage out of considering a simple example, before we add any new primitive operations, let's try to learn more about what's expressible using those we already have. With our primitives and choices of meaning for them, we've carved out a little universe for ourselves. We now get to discover what ideas are expressible in this universe. This can and should be a fluid process—we can change the rules of our universe at any time, make a fundamental change to our representation or introduce a new primitive, and explore how our creation then behaves.

Let's begin by implementing the functions of the API that we've developed so far. Now that we have a representation for `Par`, a first crack at it should be straightforward. What follows is a simplistic implementation using the representation of `Par` that we've chosen.

Listing 7.5 Basic implementation for `Par`

`unit` is represented as a function that returns a `UnitFuture`, which is a simple implementation of `Future` that just wraps a constant value. It doesn't use the `ExecutorService` at all. It's always done and can't be cancelled. Its `get` method simply returns the value that we gave it.

```
object Par {
  def unit[A](a: A): Par[A] = (es: ExecutorService) => UnitFuture(a)
```

```
  private case class UnitFuture[A](get: A) extends Future[A] {
    def isDone = true
    def get(timeout: Long, units: TimeUnit) = get
    def isCancelled = false
    def cancel(evenIfRunning: Boolean): Boolean = false
  }
```

`map2` doesn't evaluate the call to `f` in a separate logical thread, in accord with our design choice of having `fork` be the sole function in the API for controlling parallelism. We can always do `fork(map2(a, b)(f))` if we want the evaluation of `f` to occur in a separate thread.

```
  def map2[A,B,C](a: Par[A], b: Par[B])(f: (A,B) => C): Par[C] =
    (es: ExecutorService) => {
      val af = a(es)
      val bf = b(es)
      UnitFuture(f(af.get, bf.get))
    }
```

This implementation of `map2` does *not* respect timeouts. It simply passes the `ExecutorService` on to both `Par` values, waits for the results of the `Futures` `af` and `bf`, applies `f` to them, and wraps them in a `UnitFuture`. In order to respect timeouts, we'd need a new `Future` implementation that records the amount of time spent evaluating `af`, and then subtracts that time from the available time allocated for evaluating `bf`.

```
def fork[A] (a: => Par[A]): Par[A] =
  es => es.submit(new Callable[A] {
    def call = a(es).get
  })
}
```

This is the simplest and most natural implementation of `fork`, but there are some problems with it—for one, the outer `Callable` will block waiting for the “inner” task to complete. Since this blocking occupies a thread in our thread pool, or whatever resource backs the `ExecutorService`, this implies that we’re losing out on some potential parallelism. Essentially, we’re using two threads when one should suffice. This is a symptom of a more serious problem with the implementation that we’ll discuss later in the chapter.

We should note that `Future` doesn’t have a purely functional interface. This is part of the reason why we don’t want users of our library to deal with `Future` directly. But importantly, even though methods on `Future` rely on side effects, our entire `Par` API remains pure. It’s only after the user calls `run` and the implementation receives an `ExecutorService` that we expose the `Future` machinery. Our users therefore program to a pure interface whose implementation nevertheless relies on effects at the end of the day. But since our API remains pure, these effects aren’t *side* effects. In part 4 we’ll discuss this distinction in detail.



EXERCISE 7.3

Hard: Fix the implementation of `map2` so that it respects the contract of timeouts on `Future`.



EXERCISE 7.4

This API already enables a rich set of operations. Here’s a simple example: using `lazyUnit`, write a function to convert any function `A => B` to one that evaluates its result asynchronously.

```
def asyncF[A,B] (f: A => B): A => Par[B]
```

Adding infix syntax using implicit conversions

If `Par` were an actual data type, functions like `map2` could be placed in the class body and then called with infix syntax like `x.map2(y)(f)` (much like we did for `Stream` and `Option`). But since `Par` is just a type alias, we can’t do this directly. There’s a trick to add infix syntax to *any* type using *implicit conversions*. We won’t discuss that here since it isn’t that relevant to what we’re trying to cover, but if you’re interested, check out the answer code associated with this chapter.

What else can we express with our existing combinators? Let's look at a more concrete example.

Suppose we have a `Par[List[Int]]` representing a parallel computation that produces a `List[Int]`, and we'd like to convert this to a `Par[List[Int]]` whose result is sorted:

```
def sortPar(parList: Par[List[Int]]): Par[List[Int]]
```

We could of course run the `Par`, sort the resulting list, and repackage it in a `Par` with `unit`. But we want to avoid calling `run`. The only other combinator we have that allows us to manipulate the value of a `Par` in any way is `map2`. So if we passed `parList` to one side of `map2`, we'd be able to gain access to the `List` inside and sort it. And we can pass whatever we want to the other side of `map2`, so let's just pass a no-op:

```
def sortPar(parList: Par[List[Int]]): Par[List[Int]] =  
  map2(parList, unit(()))((a, _) => a.sorted)
```

That was easy. We can now tell a `Par[List[Int]]` that we'd like that list sorted. But we might as well generalize this further. We can “lift” any function of type `A => B` to become a function that takes `Par[A]` and returns `Par[B]`; we can map any function over a `Par`:

```
def map[A,B](pa: Par[A])(f: A => B): Par[B] =  
  map2(pa, unit(()))((a, _) => f(a))
```

For instance, `sortPar` is now simply this:

```
def sortPar(parList: Par[List[Int]]) = map(parList)(_.sorted)
```

That's terse and clear. We just combined the operations to make the types line up. And yet, if you look at the implementations of `map2` and `unit`, it should be clear this implementation of `map` *means* something sensible.

Was it cheating to pass a bogus value, `unit()`, as an argument to `map2`, only to ignore its value? Not at all! The fact that we can implement `map` in terms of `map2`, but not the other way around, just shows that `map2` is strictly more powerful than `map`. This sort of thing happens a lot when we're designing libraries—often, a function that seems to be primitive will turn out to be expressible using some more powerful primitive.

What else can we implement using our API? Could we map over a list in parallel? Unlike `map2`, which combines two parallel computations, `parMap` (let's call it) needs to combine *N* parallel computations. It seems like this should somehow be expressible:

```
def parMap[A,B](ps: List[A])(f: A => B): Par[List[B]]
```

We could always just write `parMap` as a new primitive. Remember that `Par[A]` is simply an alias for `ExecutorService => Future[A]`.

There's nothing wrong with implementing operations as new primitives. In some cases, we can even implement the operations more efficiently by assuming something about the underlying representation of the data types we're working with. But right now we're interested in exploring what operations are expressible using our existing

API, and grasping the relationships between the various operations we've defined. Understanding what combinators are truly primitive will become more important in part 3, when we show how to abstract over common patterns across libraries.⁵

Let's see how far we can get implementing `parMap` in terms of existing combinators:

```
def parMap[A,B](ps: List[A])(f: A => B): Par[List[B]] = {
  val fbs: List[Par[B]] = ps.map(asyncF(f))
  ...
}
```

Remember, `asyncF` converts an `A => B` to an `A => Par[B]` by forking a parallel computation to produce the result. So we can fork off our N parallel computations pretty easily, but we need some way of collecting their results. Are we stuck? Well, just from inspecting the types, we can see that we need some way of converting our `List[Par[B]]` to the `Par[List[B]]` required by the return type of `parMap`.



EXERCISE 7.5

Hard: Write this function, called `sequence`. No additional primitives are required. Do not call `run`.

```
def sequence[A](ps: List[Par[A]]): Par[List[A]]
```

Once we have `sequence`, we can complete our implementation of `parMap`:

```
def parMap[A,B](ps: List[A])(f: A => B): Par[List[B]] = fork {
  val fbs: List[Par[B]] = ps.map(asyncF(f))
  sequence(fbs)
}
```

Note that we've wrapped our implementation in a call to `fork`. With this implementation, `parMap` will return immediately, even for a huge input list. When we later call `run`, it will fork a single asynchronous computation which itself spawns N parallel computations, and then waits for these computations to finish, collecting their results into a list.



EXERCISE 7.6

Implement `parFilter`, which filters elements of a list in parallel.

```
def parFilter[A](as: List[A])(f: A => Boolean): Par[List[A]]
```

⁵ In this case, there's another good reason not to implement `parMap` as a new primitive—it's challenging to do correctly, particularly if we want to properly respect timeouts. It's frequently the case that primitive combinators encapsulate some rather tricky logic, and reusing them means we don't have to duplicate this logic.